

Session 18 (AIML) - Assignment Question

- **Name of Student :** Shruti Dnyaneshwar Darwatkar
- **Domain Name:** AIML Development
- **College Name:** Zeal Polytechnic , Narhe
- **Branch Name:** AIML
- **Github url:** https://github.com/shrutidarwatkar-sketch/Task_ITR.git

Q1. Data Loading & Initial Analysis) Load the Ford Car Dataset using pandas.

- Display the first 10 rows and last 5 rows.
- Check the shape (number of rows and columns) and data types of all columns.
- Write observations about the dataset structure in comments

PROGRAM SCREENSHOT :

```
}
0s import pandas as pd

}
9s from google.colab import files
    uploaded = files.upload()

Choose files insurance.csv
insurance.csv(text/csv) - 50264 bytes, last modified: 09/07/2026 - 100% done
Saving insurance.csv to insurance.csv
```



#Q1

```
# Load the dataset
df = pd.read_csv('/content/insurance.csv')

# Display first 10 rows
print("First 10 Rows")
print(df.head(10))

# Display last 5 rows
print("\nLast 5 Rows")
print(df.tail())

# Display shape
print("\nShape of Dataset:")
print(df.shape)

# Display data types
print("\nData Types:")
print(df.dtypes)

# Observations:
# 1. The dataset contains both numerical and categorical columns.
# 2. Each row represents an insurance customer.
# 3. 'charges' is the target variable.
```

*** First 10 Rows

	age	sex	bmi	children	smoker	region	expenses
0	19	female	27.9	0	yes	southwest	16884.92
1	18	male	33.8	1	no	southeast	1725.55
2	28	male	33.0	3	no	southeast	4449.46
3	33	male	22.7	0	no	northwest	21984.47
4	32	male	28.9	0	no	northwest	3866.86
5	31	female	25.7	0	no	southeast	3756.62
6	46	female	33.4	1	no	southeast	8240.59
7	37	female	27.7	3	no	northwest	7281.51
8	37	male	29.8	2	no	northeast	6406.41
9	60	female	25.8	0	no	northwest	28923.14

Last 5 Rows

	age	sex	bmi	children	smoker	region	expenses
1333	50	male	31.0	3	no	northwest	10600.55
1334	18	female	31.9	0	no	northeast	2205.98
1335	18	female	36.9	0	no	southeast	1629.83
1336	21	female	25.8	0	no	southwest	2007.95
1337	61	female	29.1	0	yes	northwest	29141.36

Shape of Dataset:
(1338, 7)

Data Types:

```
age      int64
sex      object
bmi      float64
children int64
smoker   object
region   object
expenses float64
dtype: object
```

Q2. Missing & Duplicate Values) Perform a detailed check on the dataset:

- Find the total number of missing values in each column.
- Identify and remove duplicate rows if any.
- Write a summary of your findings and how you handled them in comments.

PROGRAM SCREENSHOT :

```
#Q2

# Check missing values
print("Missing Values:")
print(df.isnull().sum())

# Check duplicate rows
print("\nDuplicate Rows:", df.duplicated().sum())

# Remove duplicate rows
df = df.drop_duplicates()

print("\nShape After Removing Duplicates:")
print(df.shape)

# Observation:
# Duplicate rows are removed if present.
# Missing values should be handled before model building.
```

```
... Missing Values:
age      0
sex      0
bmi      0
children 0
smoker   0
region   0
expenses 0
dtype: int64

Duplicate Rows: 1

Shape After Removing Duplicates:
(1337, 7)
```

Q3. (Statistical Summary)

Generate the statistical summary of all numeric columns using describe().

- Identify minimum, maximum, mean, and median values for key columns like price, mileage, and year.

PROGRAM SCREENSHOT :

```
#Q3
# Statistical Summary

print(df.describe())

# Minimum, Maximum, Mean and Median
columns = ['age', 'bmi', 'children', 'expenses']

for col in columns:
    print(f"\nColumn: {col}")
    print("Minimum :", df[col].min())
    print("Maximum :", df[col].max())
    print("Mean      :", df[col].mean())
    print("Median    :", df[col].median())
```

```
***
count    1338.000000    1338.000000    1338.000000    1338.000000
mean      39.207025     30.665471     1.094918    13270.422414
std       14.049960      6.098382     1.205493    12110.011240
min       18.000000     16.000000     0.000000     1121.870000
25%       27.000000     26.300000     0.000000     4740.287500
50%       39.000000     30.400000     1.000000     9382.030000
75%       51.000000     34.700000     2.000000    16639.915000
max       64.000000     53.100000     5.000000    63770.430000
```

```
Column: age
Minimum : 18
Maximum : 64
Mean    : 39.20702541106129
Median  : 39.0
```

```
Column: bmi
Minimum : 16.0
Maximum : 53.1
Mean    : 30.66547085201794
Median  : 30.4
```

```
Column: children
Minimum : 0
Maximum : 5
Mean    : 1.0949177877429
Median  : 1.0
```

```
Column: expenses
Minimum : 1121.87
Maximum : 63770.43
Mean    : 13270.422414050823
Median  : 9382.029999999999
```

Q4. Histogram of Numeric Features)

Plot histograms for all important numeric columns (price, mileage, year, engine Size, mpg).

- Analyze the distribution for each feature and note your findings in comments.

PROGRAM SCREENSHOT :

```
#Q4

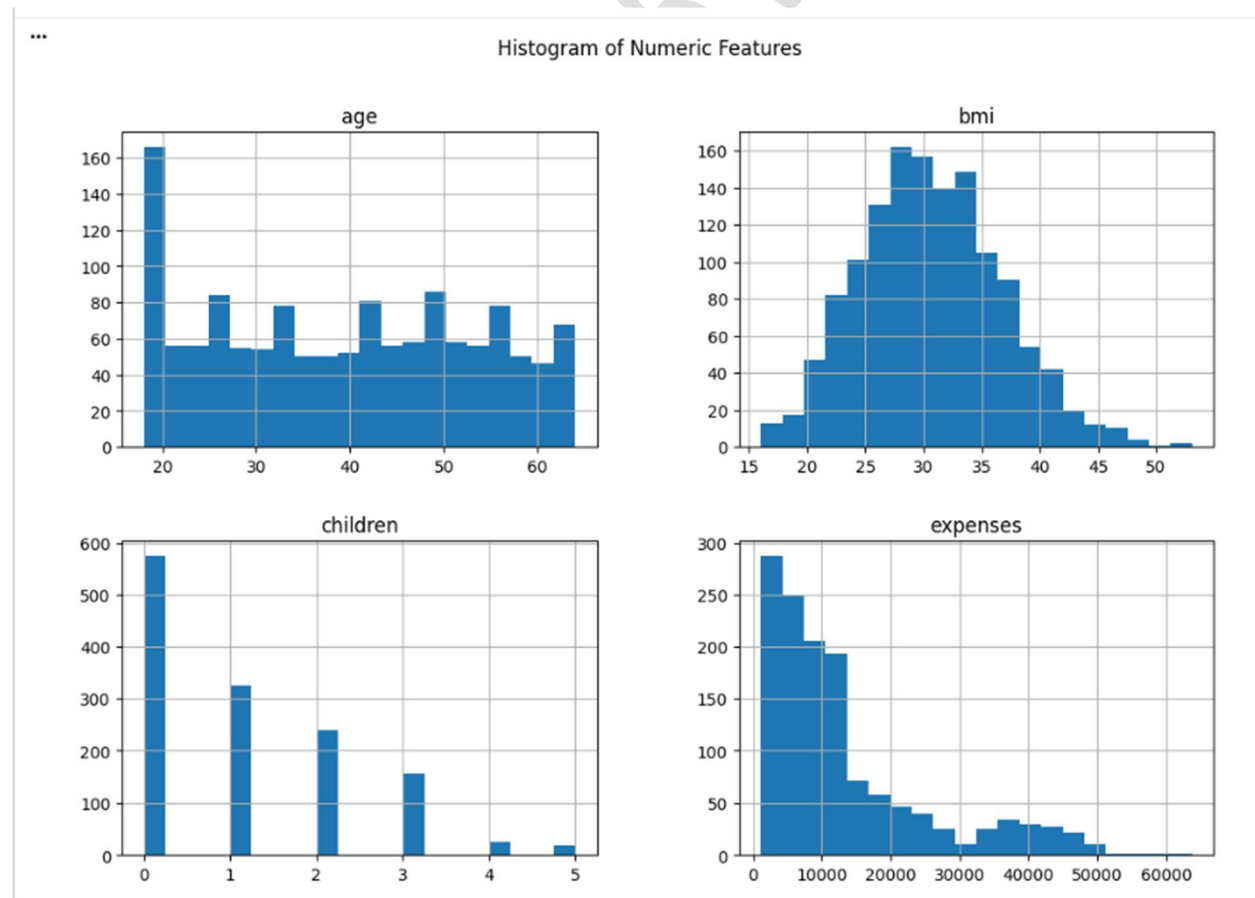
import matplotlib.pyplot as plt

numeric_columns = ['age', 'bmi', 'children', 'expenses']

df[numeric_columns].hist(figsize=(12,8), bins=20)

plt.suptitle("Histogram of Numeric Features")
plt.show()

# Observation:
# Histograms show the distribution of numerical variables.
```



Q5. Count Plots of Categorical Features)

Create count plots for all categorical columns (fuel Type, transmission, model, etc.) using seaborn.

- Analyze which categories are most common.
- Write insights about market trends based on these plots in comments.

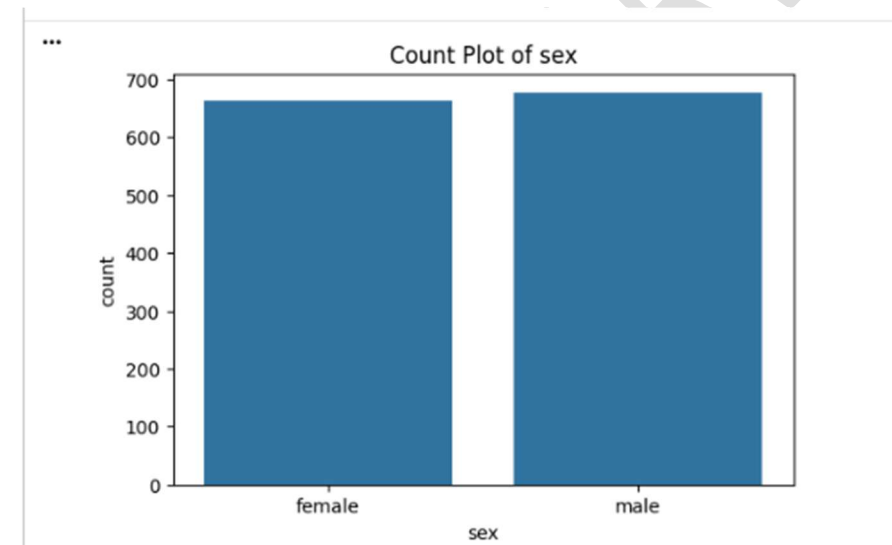
PROGRAM SCREENSHOT :

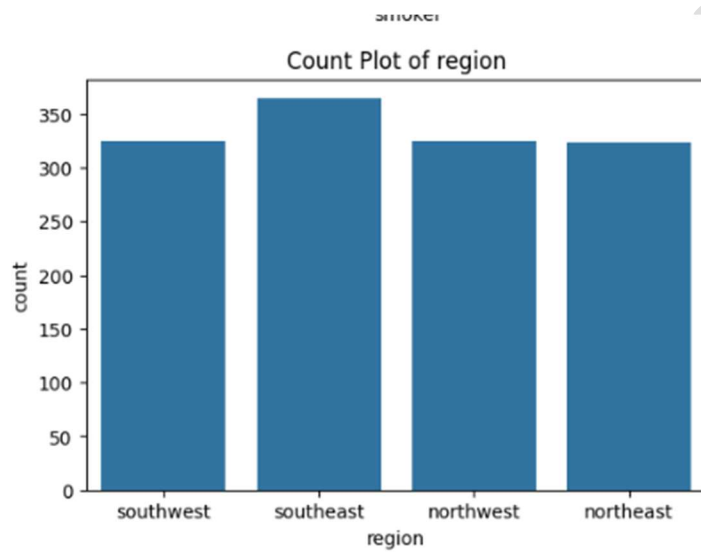
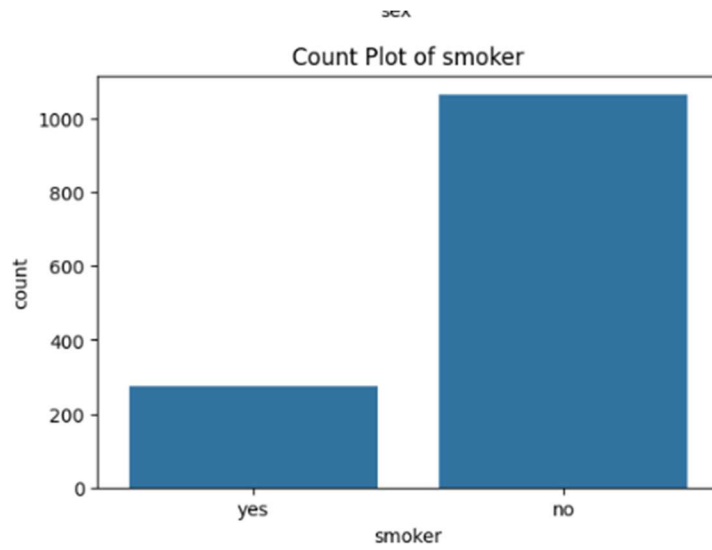
```
#Q5
import seaborn as sns
import matplotlib.pyplot as plt

categorical_columns = ['sex', 'smoker', 'region']

for col in categorical_columns:
    plt.figure(figsize=(6,4))
    sns.countplot(data=df, x=col)
    plt.title(f"Count Plot of {col}")
    plt.show()

# Observation:
# Count plots show the frequency of each category.
```





Q6. Correlation Heatmap)

Create a heatmap of the correlation matrix for numeric features.

- Use seaborn with proper annotations.
- Identify which features are highly correlated with the target variable (price).
- Write your observations in comment.

PROGRAM SCREENSHOT :

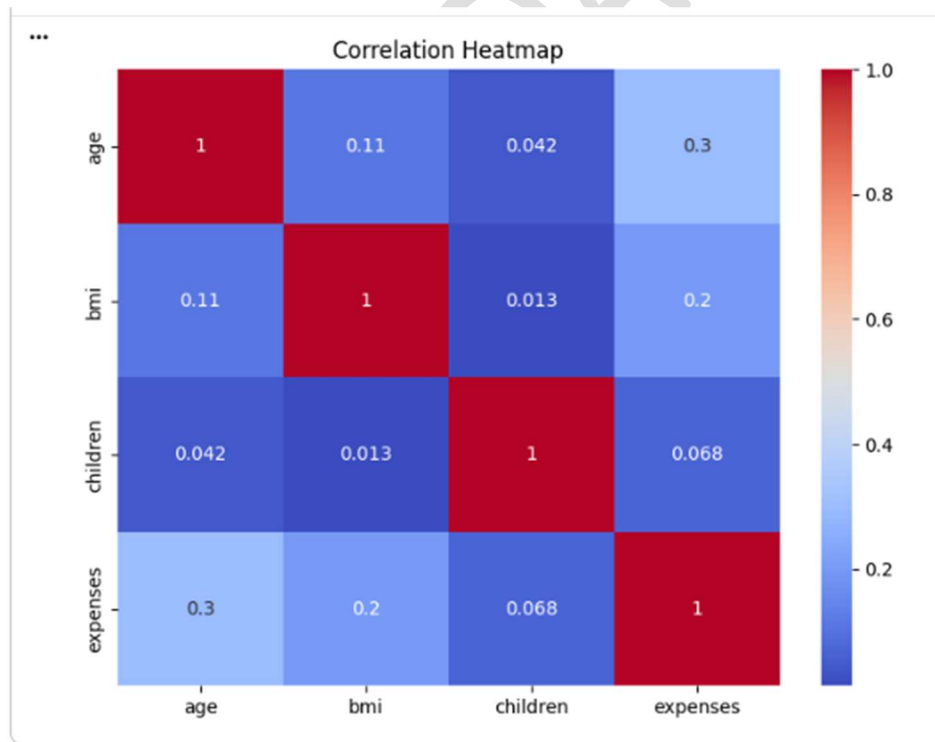
```
#Q6
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(8,6))

sns.heatmap(df.corr(numeric_only=True),
            annot=True,
            cmap='coolwarm')

plt.title("Correlation Heatmap")
plt.show()

# Observation:
# Identify features highly correlated with expenses.
```



Q7. Feature Identification) Clearly identify and list:

- Independent Features Input Features)
- Dependent Feature Output/Target Feature)

Justify your choice with reasoning.

PROGRAM SCREENSHOT :

```
#Q7

# Independent Features
X = df.drop('expenses', axis=1)

# Dependent Feature
y = df['expenses']

print("Independent Features:")
print(X.columns)

print("\nDependent Feature:")
print(y.name)

# Justification:
# 'expenses' is the output variable.
# All remaining columns are input features.
```

```
... Independent Features:
Index(['age', 'sex', 'bmi', 'children', 'smoker', 'region'], dtype='object')

Dependent Feature:
expenses
```

Q8. Encoding Categorical Variables)

Convert all categorical columns into numeric form using appropriate encoding techniques:

- Use One Hot Encoding.
- Show before and after transformation for at least 2 columns.

PROGRAM SCREENSHOT :

```
#Q8

# Display Before Encoding
print("Before Encoding")
print(df[['sex', 'smoker', 'region']].head())

# One Hot Encoding
encoded_df = pd.get_dummies(df, drop_first=True)

print("\nAfter Encoding")
print(encoded_df.head())

# Observation:
# Categorical variables are converted into numeric variables.
```

```
... Before Encoding
      sex smoker  region
0  female    yes southwest
1   male     no  southeast
2   male     no  southeast
3   male     no northwest
4   male     no northwest

After Encoding
   age  bmi  children  expenses  sex_male  smoker_yes  region_northwest \
0  19  27.9         0  16884.92    False         True             False
1  18  33.8         1   1725.55     True         False             False
2  28  33.0         3   4449.46     True         False             False
3  33  22.7         0  21984.47     True         False              True
4  32  28.9         0   3866.86     True         False              True

   region_southeast  region_southwest
0              False                True
1              True                False
2              True                False
3              False                False
4              False                False
```

Q9. Feature Scaling)

Apply Standard Scaling (using StandardScaler from sklearn) on the numeric independent features

- . Show the first 5 rows of scaled data.

PROGRAM SCREENSHOT :

```
#Q9

from sklearn.preprocessing import StandardScaler

# Input Features
X = df.drop('expenses', axis=1)

# One Hot Encoding
X = pd.get_dummies(X, drop_first=True)

# Numerical Columns
numeric_columns = ['age', 'bmi', 'children']

# Standard Scaling
scaler = StandardScaler()

X[numeric_columns] = scaler.fit_transform(X[numeric_columns])

print("Scaled Data")
print(X.head())
```

```
... Scaled Data
   age      bmi  children  sex_male  smoker_yes  region_northwest \
0 -1.438764 -0.453646 -0.908614    False         True             False
1 -1.509965  0.514186 -0.078767     True         False             False
2 -0.797954  0.382954  1.580926     True         False             False
3 -0.441948 -1.306650 -0.908614     True         False              True
4 -0.513149 -0.289606 -0.908614     True         False              True

   region_southeast  region_southwest
0             False                True
1              True                False
2              True                False
3             False                False
4             False                False
```

Q10. Mini Project - Complete Preprocessing Pipeline)

Create a complete preprocessing pipeline for the Ford Car Dataset:

1. Load and clean the data (handle missing & duplicate values).
2. Perform EDA (histograms + count plots + heatmap).
3. Identify input and output features.
4. Encode categorical variables.

PROGRAM SCREENSHOT :

```
#Q10

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler

# Load Dataset
data = pd.read_csv('/content/insurance.csv')

# Remove Duplicates
data.drop_duplicates(inplace=True)

# Missing Values
print(data.isnull().sum())

# Histograms
numeric_columns = ['age', 'bmi', 'children', 'expenses']

data[numeric_columns].hist(figsize=(12,8), bins=20)
plt.suptitle("Histograms")
plt.show()

# Count Plots
categorical_columns = ['sex', 'smoker', 'region']

for col in categorical_columns:
    plt.figure(figsize=(6,4))
    sns.countplot(data=data, x=col)
    plt.title(col)
    plt.show()

# Correlation Heatmap
plt.figure(figsize=(8,6))
sns.heatmap(data.corr(numeric_only=True),
            annot=True,
            cmap='coolwarm')
plt.title("Correlation Heatmap")
plt.show()
```

```

# Input and Output Features
X = data.drop('expenses', axis=1)
y = data['expenses']

# One Hot Encoding
X = pd.get_dummies(X, drop_first=True)

# Feature Scaling
numeric_columns = ['age', 'bmi', 'children']

scaler = StandardScaler()

X[numeric_columns] = scaler.fit_transform(X[numeric_columns])

print("Processed Features")
print(X.head())

print("\nTarget Variable")
print(y.head())

print("\nPreprocessing Completed Successfully.")

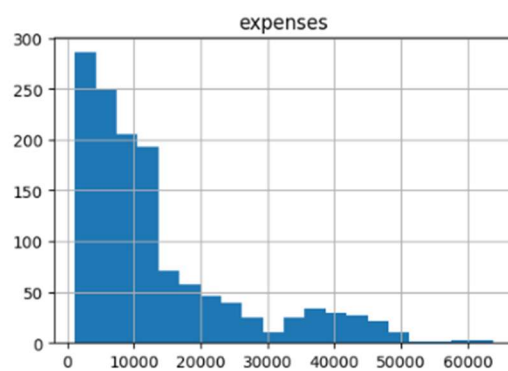
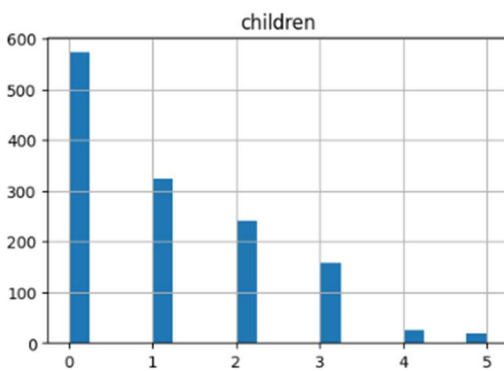
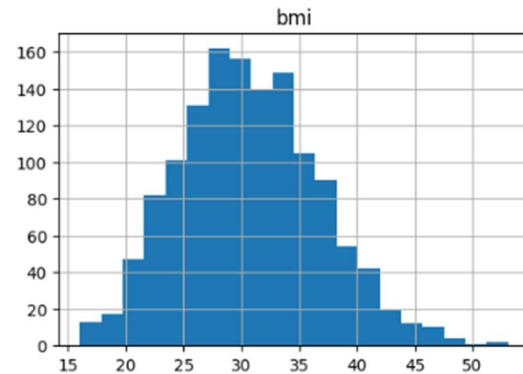
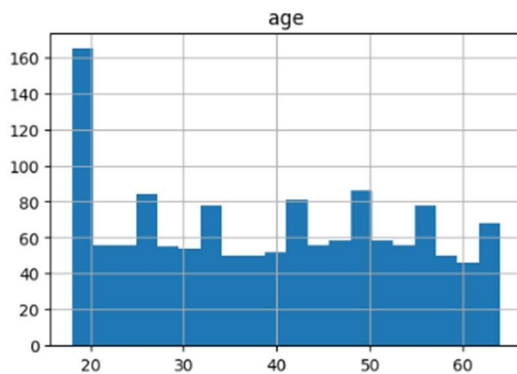
```

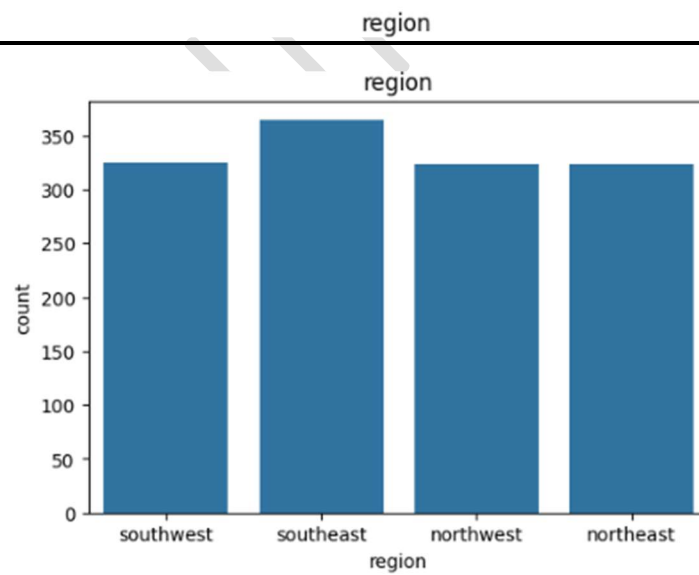
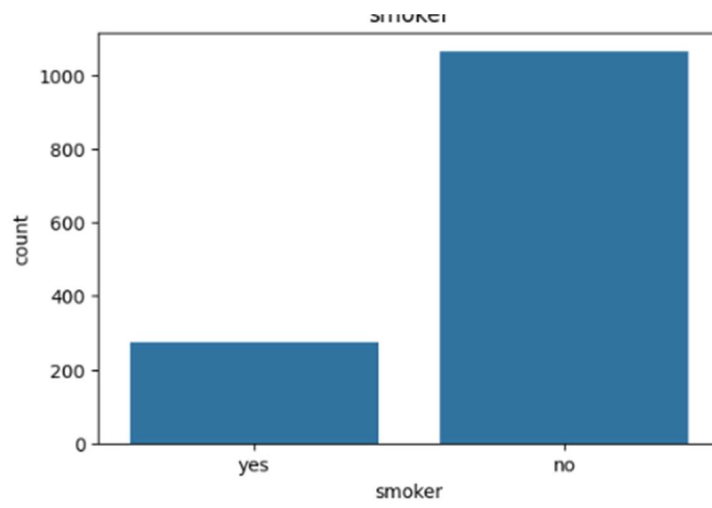
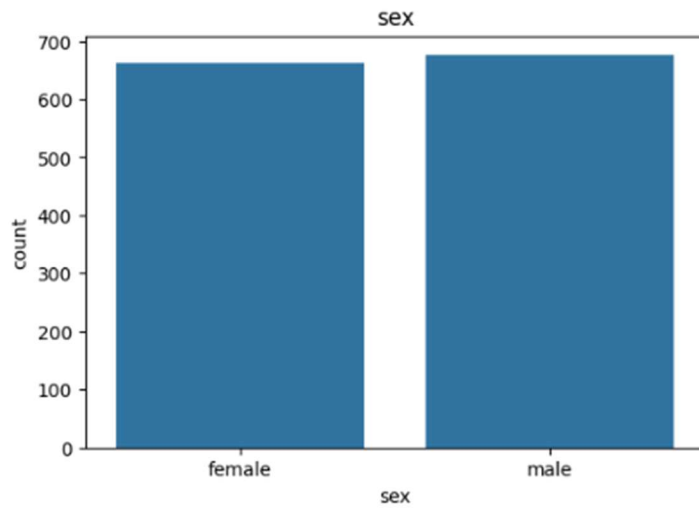
```

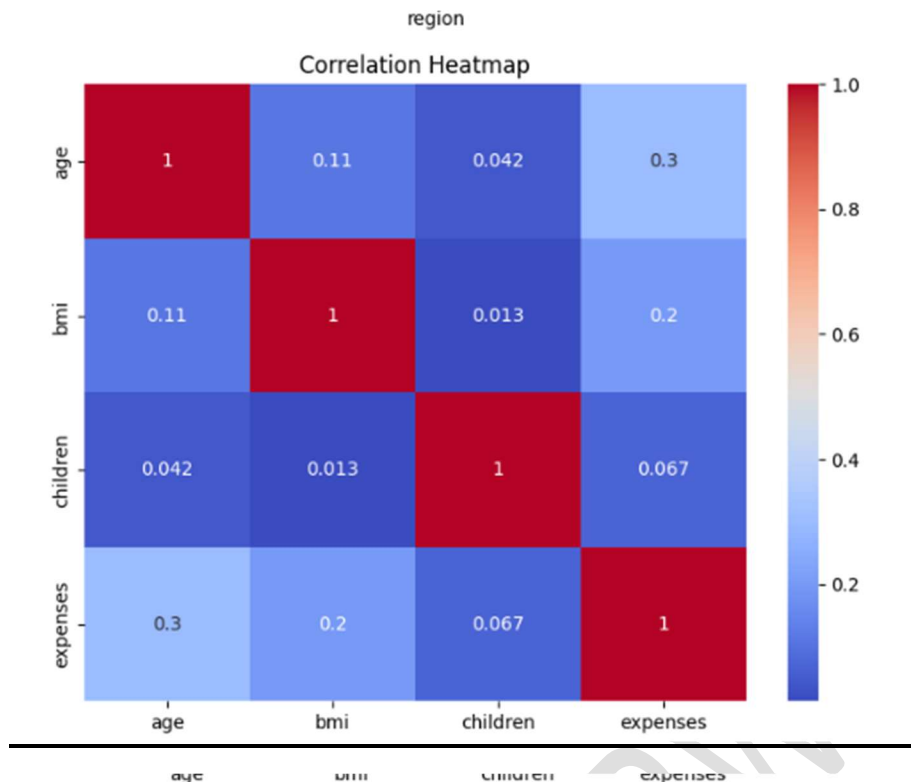
... age      0
    sex      0
    bmi      0
    children  0
    smoker   0
    region   0
    expenses  0
    dtype: int64

```

Histograms







```

Processed Features
  age      bmi  children  sex_male  smoker_yes  region_northwest \
0 -1.440418 -0.453484 -0.909234   False        True           False
1 -1.511647  0.513986 -0.079442    True        False           False
2 -0.799350  0.382803  1.580143    True        False           False
3 -0.443201 -1.306169 -0.909234    True        False            True
4 -0.514431 -0.289506 -0.909234    True        False            True

```

```

  region_southeast  region_southwest
0             False                True
1              True                False
2              True                False
3             False                False
4             False                False

```

```

Target Variable
0    16884.92
1     1725.55
2     4449.46
3    21984.47
4     3866.86
Name: expenses, dtype: float64

```

Preprocessing Completed Successfully.

SHRUTI DARWATKAR